```
/img/pdf_icon.png
/titles/rails7/agile-web-development-with-rails-7/rails7-beta-125.jpg
```

This example could be improved significantly. In particular, it doesn't do much in the way of error handling. It should really report "Not Found" errors (the infamous 404) and should handle redirects (which happen when a web server gives the client an alternative address for the requested page).

We can take this to a higher level still. By bringing the open-uri library into a program, the URI.open method recognizes http:// and ftp:// URLs in the filename. Not just that—it also handles redirects automatically.

tut_io/networking_2.rb

```
require "open-uri"

URI.open("https://pragprog.com/titles/ruby5/programming-ruby-3-2-5th-edition/") do |f|
  puts f.read.scan(/<img class=".*?" src="(.*?)"/m).uniq[0,3]
end
```

produces:

```
/titles/ruby5/programming-ruby-3-2-5th-edition/ruby5-beta-250.jpg
/img/pdf_icon.png
/titles/rails7/agile-web-development-with-rails-7/rails7-beta-125.jpg
```

What's Next

We've seen both Ruby's simple I/O library, implemented as a series of methods in the Kernel module, and the more complicated I/O methods in the class IO and its children. We've seen how to read and write data.

A common problem with I/O is that it is slow and blocks programs. A common workaround is to use threading to allow the program to do multiple things at once. Let's take a look at some of Ruby's threading options.